

Saketh Koona

📞 (470)-398-1715 ✉️ skoona3@gatech.edu 🔗 <https://www.linkedin.com/in/skoona3/>

Education

Georgia Institute of Technology

BS in Industrial Engineering

May 2026

Atlanta, GA

- **Relevant Coursework:** Data Structures & Algorithms, Machine Learning, Database Systems, Computer Vision

Technical Skills

Languages: Python, Elixir, SQL, Rust, TypeScript, C, C++, R, Java, Go

Technologies: Next.js, Git, Apache Spark, Apache Airflow, Kafka, OpenAI Gym, LangChain, AWS, Azure, GCP, Pandas, NumPy, Postgres

Experience

Software Engineer Intern

Transpira Labs

Jul 2026 - Present

Atlanta, GA

- Engineered core GTM platform with in-browser **WebRTC** softphone and **webhook** pipelines, automating outbound lead acquisition
- Architected an AI quoting engine parsing multi-channel streams (**Gmail, WhatsApp, Twilio, SMS**) via **LLM Tool Use**
- Built **DAT** and **Truckstop** ingestion and dispatch pipelines using **Next.js** and **Neon Postgres**
- Constructed **RLVR** environments and deterministic verifiers for reasoning benchmarks including **GDPVal, AccountingBench**

Open Source Contributor

NeMo-RL

Jun 2026 - Present

Atlanta, GA

- Contributed to **NVIDIA NeMo-RL** (open source), adding **800+** lines of unit and functional tests covering untested CLI entry points for checkpoint converters

Lead Data Chair

Joint Finance Data Committee @ Georgia Tech

Apr 2025 – Dec 2025

Atlanta, GA

- Leading a team of **15** in building and maintaining financial data systems to track **over 450** student organizations.
- Designed and deployed a **PostgreSQL** architecture on **AWS RDS** to manage over **\$25M** in annual transactions.
- Applied **XGBoost** and computer vision feature extraction to forecast and audit suspicious transactions, saving **\$13K**.
- Developed and maintained **ETL** workflows using **Apache Airflow** and **SQL** on **AWS EC2**, processing **12k** transactions monthly.

Machine Learning Student Researcher

Structured Information for Precision Neuroengineering Lab (SIPLab)

Jan 2025 – May 2026

Atlanta, GA

- Built custom **state-space architecture** in **PyTorch** to model latent brain dynamics across **256** neural channels.
- Trained **GRU-based recurrent networks** to simulate opsin behavior and capture nonlinear control dynamics, with **2%** reconstruction error.
- Generated seeded synthetic dataset pipeline with **Cleo** and **NumPy** for system identification, with **95%** eigenvalue recovery.
- Orchestrated large-scale, GPU-accelerated experiments via **Snakemake** and **SLURM** on HPC clusters with **H200 GPUs**.

Projects

Distributed Image Processing Engine | Rust, Kafka, Axum, Docker, AWS S3, MongoDB, Tokio

- Architected an event-driven **Rust** microservices pipeline using **Kafka** to asynchronously process large image workloads
- Decoupled ingress via an **Axum** API gateway with horizontally scaled, stateless worker pools using consumer groups
- Built a concurrent, topological **DAG scheduler** in **Tokio** to orchestrate multi-stage transformation tasks
- Implemented fault-tolerant persistence with **at-least-once processing** and idempotent writes to **AWS S3** and **MongoDB**

Build: Scratch for Reinforcement Learning (1st Place, YC RL Hackathon) | Python, HUD, React, FastAPI

- Won **1st place** at the **HUD x YC RSI RL Hackathon**, earning a guaranteed **YC F26** interview.
- Built a **drag-and-drop** canvas compiling Environment/Tool/Task/Train blocks into runnable **RL environments**
- Built and shipped runnable templates spanning **tool use, reasoning, and classification** tasks, letting users build and train agents end-to-end.
- Turned the builder into its own **RL environment**, measuring agents' ability to compile block code into runnable **RL envs**.

C++ Market Exchange Simulator | C++

- Built a low-latency **price-time priority matching engine** and **internal orderbook**, supporting multiple order types.
- Implemented a lock-free **event-driven architecture** emitting deterministic order and trade events
- Built a **binary order entry protocol (BOE/OUCH-style)** with fixed-width messages, sequencing, and explicit acknowledgments
- Accepted order ingress via **TCP sessions** and published market data egress via **UDP multicast**
- Processed **12,000+** orders in single-threaded symbol partitions under synthetic load
- Optimized hot-path performance using **arena-based memory allocation** and object pooling